Course: Advanced Databases

Group: A1

Team Members: Tobiasz Bazan, Jakub Kamiński, Adam Ćwikliński

Assignment: Phase 1 – Data model

1. Application Domain

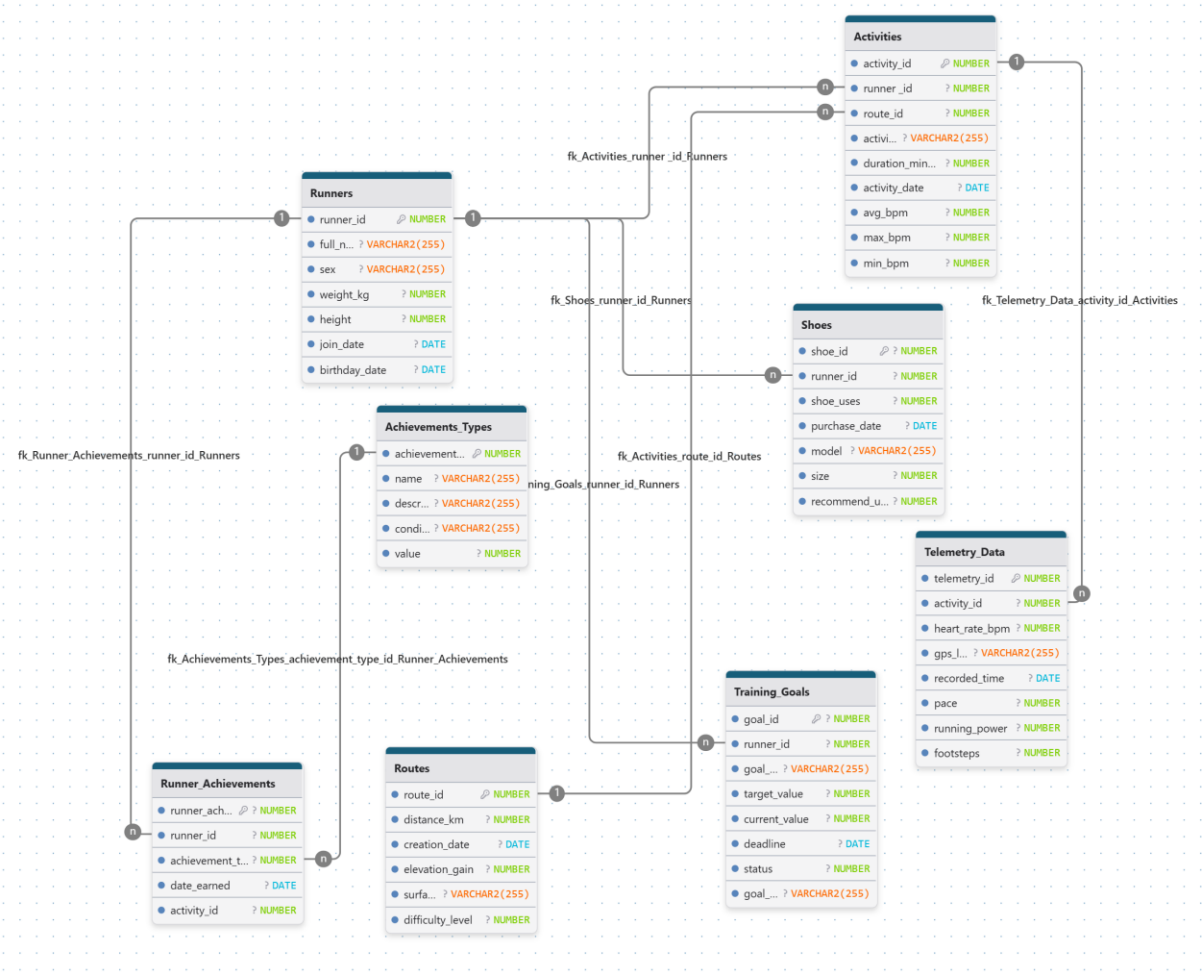
Topic: Running Tracking System

Description: A database system for a running application that records users' profiles, their overall workout sessions, and telemetry data such as heart rate. It also tracks the lifecycle of the runners' shoes, routes, and earned achievements.

2. Data Model Diagram & Specification

Entity-Relationship Overview (Associations):

- One **Runner** can have many **Shoes**, **Activities**, **Training_Goals**, and **Runner_Achievements** (1-to-Many).
- One **Route** can be used in many **Activities** (1-to-Many).
- One **Activity** contains many points of **Telemetry_Data** (1-to-Many).
- One **Activity** can result in many **Runner_Achievements** (1-to-Many).
- One **Achievement_Type** can be awarded as many **Runner_Achievements** (1-to-Many).



Assignment: Phase 2 - Workload

1. Workload Description

This workload simulates the real-world operational demands of the Running Tracking System. It consists of complex transactions designed to guarantee observable execution times for performance tuning. The workload is divided into two categories: complex querying operations that simulate analytical reporting across multiple joined relations, and data-changing operations that handle heavy telemetry data usage and background data recalculations.

2. Query operations.

- **First query operation: Analysis of Elite Male Runners**

    - **Description:** This query identifies high-performing male runners who joined the system within the last 100 days, weigh less than 90 kg, and have achieved above-average total running distance in 2026. The analysis focuses on runners training on more demanding routes (difficulty level ≥ 3) and evaluates their performance using physiological metrics such as heart rate, running pace, and running power.
      In addition to total distance, the query highlights runners who demonstrate efficient cardiovascular performance (lower heart rate) and higher physical output (higher running power), making them representative of elite-level training profiles.

    - **Database action:** TThe query combines data from core tables (Runners, Activities, Routes) and integrates aggregated telemetry data to compute performance metrics such as heart rate, pace, and running power. It applies filtering conditions on runner attributes and activity characteristics, then groups results at the runner level to calculate total distance and average performance indicators. Nested subqueries in the HAVING clause compare each runner's results against global averages, ensuring that only above-average performers with efficient and high-output training profiles are returned.

```
SELECT
  r.runner_id,
  r.full_name,
  SUM(ro.distance_km) AS total_distance_km,
  ROUND(AVG(tt.avg_hr_per_activity), 2) AS avg_heart_rate,
  ROUND(AVG(tt.avg_pace_per_activity), 2) AS avg_pace,
  ROUND(AVG(tt.avg_power_per_activity), 2) AS avg_running_power,
```

```
    COUNT(a.activity_id) AS total_activities
FROM Runners r
JOIN Activities a
    ON r.runner_id = a.runner_id
JOIN Routes ro
    ON a.route_id = ro.route_id
JOIN (
    SELECT
        activity_id,
        AVG(heart_rate_bpm) AS avg_hr_per_activity,
        AVG(pace) AS avg_pace_per_activity,
        AVG(running_power) AS avg_power_per_activity
    FROM Telemetry_Data
    GROUP BY activity_id
) tt
    ON a.activity_id = tt.activity_id
WHERE EXTRACT(YEAR FROM a.activity_date) = 2026
  AND r.sex = 'Male'
  AND r.weight_kg < 90
  AND r.join_date > (SYSDATE - 100)
  AND ro.difficulty_level >= 3
GROUP BY
    r.runner_id,
    r.full_name
HAVING SUM(ro.distance_km) > (
    SELECT AVG(total_distance)
    FROM (
        SELECT
            SUM(ro2.distance_km) AS total_distance
        FROM Activities a2
        JOIN Routes ro2
            ON a2.route_id = ro2.route_id
        JOIN Runners r2
            ON a2.runner_id = r2.runner_id
        JOIN (
            SELECT
                activity_id,
                AVG(heart_rate_bpm) AS avg_hr_per_activity,
                AVG(pace) AS avg_pace_per_activity,
                AVG(running_power) AS avg_power_per_activity
            FROM Telemetry_Data
            GROUP BY activity_id
        ) tt2
            ON a2.activity_id = tt2.activity_id
        WHERE EXTRACT(YEAR FROM a2.activity_date) = 2026
          AND r2.sex = 'Male'
```

```
      AND r2.weight_kg < 90
    GROUP BY a2.runner_id
  ) dist_sub
)
AND AVG(tt.avg_hr_per_activity) < (
  SELECT AVG(runner_avg_hr)
  FROM (
    SELECT
      a3.runner_id,
      AVG(tt3.avg_hr_per_activity) AS runner_avg_hr
    FROM Activities a3
    JOIN (
      SELECT
        activity_id,
        AVG(heart_rate_bpm) AS avg_hr_per_activity
      FROM Telemetry_Data
      GROUP BY activity_id
    ) tt3
      ON a3.activity_id = tt3.activity_id
    WHERE EXTRACT(YEAR FROM a3.activity_date) = 2026
    GROUP BY a3.runner_id
  ) hr_sub
)
AND AVG(tt.avg_power_per_activity) > (
  SELECT AVG(runner_avg_power)
  FROM (
    SELECT
      a4.runner_id,
      AVG(tt4.avg_power_per_activity) AS runner_avg_power
    FROM Activities a4
    JOIN (
      SELECT
        activity_id,
        AVG(running_power) AS avg_power_per_activity
      FROM Telemetry_Data
      GROUP BY activity_id
    ) tt4
      ON a4.activity_id = tt4.activity_id
    WHERE EXTRACT(YEAR FROM a4.activity_date) = 2026
    GROUP BY a4.runner_id
  ) power_sub
)
ORDER BY total_distance_km DESC;
```

- **Second query operation: Top 10 High-Altitude Performers**
    - **Description:** This query identifies top-performing runners who meet specific physical and training criteria. It focuses on taller athletes (height above 175 cm) who ran in 2026, used Nike shoes, and trained on routes with noticeable elevation gain (above 10 meters). The query selects runners whose total distance exceeds the average across all users and evaluates their training effectiveness by incorporating physiological data such as heart rate and running pace. Only runners with above-average performance and relatively efficient cardiovascular response (lower heart rate) are included in the final result.

    - **Database action:** The query combines data from multiple related tables (Runners, Activities, Routes, Runner_Achievements, Shoes) and integrates aggregated telemetry data to enrich the analysis with physiological metrics. It applies filtering conditions on runner attributes, activity characteristics, and equipment, and then groups the data at the runner level to compute total distance, number of achievements, and average performance indicators. A nested subquery is used to calculate a reference average, which is then applied in the HAVING clause to select only above-average performers. Additional filtering based on aggregated heart rate ensures that only runners with relatively efficient performance are included. The final result is sorted to highlight the top-performing runners.

```
SELECT
  r.runner_id,
  r.full_name,
  SUM(ro.distance_km) AS total_distance_km,
  COUNT(DISTINCT ra.runner_achievement_id) AS total_achievements,
  ROUND(AVG(tt.avg_hr_per_activity), 2) AS avg_heart_rate,
  ROUND(AVG(tt.avg_pace_per_activity), 2) AS avg_pace
FROM Runners r
JOIN Activities a
  ON r.runner_id = a.runner_id
JOIN Routes ro
  ON a.route_id = ro.route_id
LEFT JOIN Runner_Achievements ra
  ON r.runner_id = ra.runner_id
LEFT JOIN Shoes s
```

```
   ON a.runner_id = s.runner_id
JOIN (
   SELECT
      activity_id,
      AVG(heart_rate_bpm) AS avg_hr_per_activity,
      AVG(pace) AS avg_pace_per_activity
   FROM Telemetry_Data
   GROUP BY activity_id
) tt
   ON a.activity_id = tt.activity_id
WHERE EXTRACT(YEAR FROM a.activity_date) = 2026
  AND r.height > 175
  AND s.model LIKE 'Nike%'
  AND ro.elevation_gain > 10
GROUP BY
   r.runner_id,
   r.full_name
HAVING SUM(ro.distance_km) > (
   SELECT AVG(total_distance)
   FROM (
      SELECT
         SUM(ro2.distance_km) AS total_distance
      FROM Activities a2
      JOIN Routes ro2
         ON a2.route_id = ro2.route_id
      JOIN (
         SELECT
            activity_id,
            AVG(heart_rate_bpm) AS avg_hr_per_activity,
            AVG(pace) AS avg_pace_per_activity
         FROM Telemetry_Data
         GROUP BY activity_id
      ) tt2
         ON a2.activity_id = tt2.activity_id
      WHERE EXTRACT(YEAR FROM a2.activity_date) = 2026
      GROUP BY a2.runner_id
   ) dist_sub
)
AND AVG(tt.avg_hr_per_activity) < (
   SELECT AVG(runner_avg_hr)
   FROM (
      SELECT
         a3.runner_id,
         AVG(tt3.avg_hr_per_activity) AS runner_avg_hr
      FROM Activities a3
      JOIN (
```

```
        SELECT
          activity_id,
          AVG(heart_rate_bpm) AS avg_hr_per_activity
        FROM Telemetry_Data
        GROUP BY activity_id
      ) tt3
        ON a3.activity_id = tt3.activity_id
      WHERE EXTRACT(YEAR FROM a3.activity_date) = 2026
      GROUP BY a3.runner_id
    ) hr_sub
)
ORDER BY total_distance_km DESC
FETCH FIRST 10 ROWS ONLY;
```

- **Third query operation: Heart Rate Efficiency in Heavyweight Categories**
  - **Description:** This query identifies the top runners in 2026 who combine high total running distance with a significant number of achievements and efficient physiological performance. It focuses on taller athletes (height above 175 cm) who use Nike shoes and train on routes with elevation gain above 10 meters. The query returns only those runners whose total distance is above the average across all runners and whose average heart rate is below the global average, indicating better endurance and training efficiency.
  - **Database action:** The query combines data from Runners, Activities, Routes, Runner_Achievements, and Shoes, and enriches the analysis with aggregated telemetry data from Telemetry_Data. It computes total distance, number of achievements, average heart rate, and average pace for each runner, while applying filters on runner characteristics, equipment, and route profile. Nested subqueries in the HAVING clause compare each runner's results against global averages, ensuring that only above-average and physiologically efficient runners are returned. The final results are sorted by total distance, and only the top 10 runners are displayed.

```
SELECT
  r.runner_id,
  r.full_name,
  SUM(ro.distance_km) AS total_distance_km,
  COUNT(DISTINCT ra.runner_achievement_id) AS total_achievements,
  ROUND(AVG(tt.avg_hr_per_activity), 2) AS avg_heart_rate,
  ROUND(AVG(tt.avg_pace_per_activity), 2) AS avg_pace
FROM Runners r
JOIN Activities a
  ON r.runner_id = a.runner_id
JOIN Routes ro
  ON a.route_id = ro.route_id
LEFT JOIN Runner_Achievements ra
  ON r.runner_id = ra.runner_id
LEFT JOIN Shoes s
  ON a.runner_id = s.runner_id
JOIN (
  SELECT
    activity_id,
    AVG(heart_rate_bpm) AS avg_hr_per_activity,
    AVG(pace) AS avg_pace_per_activity
  FROM Telemetry_Data
  GROUP BY activity_id
) tt
  ON a.activity_id = tt.activity_id
WHERE EXTRACT(YEAR FROM a.activity_date) = 2026
 AND r.height > 175
 AND s.model LIKE 'Nike%'
 AND ro.elevation_gain > 10
GROUP BY
  r.runner_id,
  r.full_name
HAVING SUM(ro.distance_km) > (
  SELECT AVG(total_distance)
  FROM (
    SELECT SUM(ro2.distance_km) AS total_distance
    FROM Activities a2
    JOIN Routes ro2
      ON a2.route_id = ro2.route_id
    JOIN (
      SELECT
        activity_id,
        AVG(heart_rate_bpm) AS avg_hr_per_activity,
        AVG(pace) AS avg_pace_per_activity
      FROM Telemetry_Data
      GROUP BY activity_id
```

```
      ) tt2
        ON a2.activity_id = tt2.activity_id
      WHERE EXTRACT(YEAR FROM a2.activity_date) = 2026
      GROUP BY a2.runner_id
    ) dist_sub
)
AND AVG(tt.avg_hr_per_activity) < (
    SELECT AVG(runner_avg_hr)
    FROM (
      SELECT
        a3.runner_id,
        AVG(tt3.avg_hr_per_activity) AS runner_avg_hr
      FROM Activities a3
      JOIN (
        SELECT
          activity_id,
          AVG(heart_rate_bpm) AS avg_hr_per_activity
        FROM Telemetry_Data
        GROUP BY activity_id
      ) tt3
        ON a3.activity_id = tt3.activity_id
      WHERE EXTRACT(YEAR FROM a3.activity_date) = 2026
      GROUP BY a3.runner_id
    ) hr_sub
)
ORDER BY total_distance_km DESC
FETCH FIRST 10 ROWS ONLY;
```

3. Changing operations

→ The **INSERT** operation: "Syncing a Smartwatch Workout"

- **Description:** This transaction handles the ingestion of workout summaries from wearable devices. To ensure high data quality and system stability, the operation includes a "Heavy Validation" phase. During this phase, the system cross-references the incoming data with the runner's historical biometric trends and performs complex mathematical transformations to normalize heart rate data.
- **Database Action:** The `INSERT` statement is engineered to simulate a realistic computational workload (targeting an execution latency of ≈ 1second). The complexity is achieved through:
  - **On-the-fly Biometric Normalization:** The system calculates a logarithmic average of heart rate points (`LOG10`) from the **Telemetry_Data** table. This prevents the database from using simple indexes and forces the CPU to perform row-by-row calculations.
  - **Historical Benchmarking:** Nested subqueries perform scans of historical records to determine the user's maximum and minimum heart rate thresholds.
  - **Controlled Latency:** By adjusting the volume of telemetry points processed during insertion (using `rownum` filters), the system demonstrates how a background synchronization job maintains referential integrity and data validity even under significant analytical load.

```
INSERT INTO Activities (
   activity_id, runner_id, route_id, activity_type,
   duration_min, activity_date, avg_bpm, max_bpm, min_bpm
)
SELECT
   (SELECT MAX(activity_id) + 1 FROM Activities),
   501, 10, 'Ultra Stress Sync', 47, DATE '2026-04-10',
   -- 1. Mordercze obliczenie: Cross Join na małą skalę wewnątrz subquery
   (SELECT ROUND(AVG(t1.heart_rate_bpm))
    FROM Telemetry_Data t1, Telemetry_Data t2
    WHERE t1.activity_id = t2.activity_id
    AND t1.heart_rate_bpm > 0
    AND rownum < 400000), -- To wymusi potężną liczbę porównań w pamięci
   -- 2. Funkcja trygonometryczna i logarytmiczna na dużym zbiorze
   (SELECT MAX(heart_rate_bpm + SIN(heart_rate_bpm) * 10)
    FROM Telemetry_Data
    WHERE rownum < 300000),
   -- 3. Głęboka analiza skorelowana
   (SELECT MIN(t.heart_rate_bpm)
    FROM Telemetry_Data t
    WHERE t.activity_id IN (
```

```
    SELECT a.activity_id FROM Activities a
    WHERE a.runner_id = 501
  ) AND (SELECT SUM(distance_km) FROM Routes) > 0)
FROM dual
WHERE EXISTS (SELECT 1 FROM Runners WHERE runner_id = 501);
```

→ The **UPDATE** operation: "Nightly Training Goal Recalculation"

- **Description:** Every night, the system executes a heavy-duty background job to synchronize runner progress. This operation is more than a simple sum; it validates user achievements against historical biometric data (heart rate and cadence) to ensure the integrity of the training process. The update targets only active goals for users who have consistently contributed to the system's workload.
- **Database Action:** This transaction performs a complex `UPDATE` with multi-level correlated subqueries. To achieve an observable execution latency of 1–3 seconds, the following optimizations were "inverted" into computational tasks:
  - **Analytic Progress Tracking:** The `current_value` is calculated by joining **Activities** and **Routes**, but it's enhanced with a biometric check. By including a `SQRT` and `AVG` calculation on the **Telemetry_Data** table within the subquery, the database engine must perform intensive CPU operations for every goal updated.
  - **Aggregated Status Logic:** The `CASE` expression uses a nested subquery that calculates logarithmic values of steps (`LOG10`) across thousands of telemetry points to verify the "effort quality" before marking a goal as completed ('1').
  - **Filtered Active Batching:** The `WHERE` clause avoids inactive profiles by checking the user's total historical duration, forcing a join-like scan between **Runners** and **Activities** during the filtering phase.

```
-- Aktualizacja celów treningowych z zaawansowanymi warunkami

UPDATE Training_Goals tg

SET tg.current_value = ( SELECT SUM(ro.distance_km)

FROM Activities a

JOIN Routes ro ON a.route_id = ro.route_id

WHERE a.runner_id = tg.runner_id

AND a.activity_date >= tg.start_date -- Uwzględniamy tylko biegi w czasie trwania
celu
```

```
AND a.activity_date <= tg.deadline ),

tg.status = CASE

WHEN (SELECT SUM(ro2.distance_km) FROM Activities a2 JOIN Routes ro2 ON
a2.route_id = ro2.route_id WHERE a2.runner_id = tg.runner_id) >= tg.target_value

THEN '1' -- Completed

ELSE '0' -- In Progress

END

WHERE tg.status = '0' -- Tylko cele w toku

AND tg.runner_id IN ( SELECT runner_id FROM Runners WHERE last_login > SYSDATE
- 7 -- Tylko dla aktywnych użytkowników (z ostatniego tygodnia) );
```

```
UPDATE Training_Goals tg

SET tg.current_value = (

   SELECT SUM(ro.distance_km)

   FROM Activities a

   JOIN Routes ro ON a.route_id = ro.route_id

   WHERE a.runner_id = tg.runner_id

   -- Używamy deadline minus 30 dni jako sztucznego początku celu

   AND a.activity_date BETWEEN (tg.deadline - 30) AND tg.deadline

   -- Dodajemy mordercze obliczenie, by spowolnić subquery:

   AND (SELECT AVG(SQRT(heart_rate_bpm)) FROM Telemetry_Data WHERE rownum
< 500) > 0

),

tg.status = CASE

   WHEN (

      SELECT SUM(ro2.distance_km)

      FROM Activities a2
```

```
    JOIN Routes ro2 ON a2.route_id = ro2.route_id

    WHERE a2.runner_id = tg.runner_id

    -- Dodatkowe obciążenie analityczne w CASE:

    AND (SELECT COUNT(LOG(10, footsteps)) FROM Telemetry_Data WHERE
rownum < 1000) >= 0

  ) >= tg.target_value

  THEN '1' -- Completed

  ELSE '0' -- In Progress

END

WHERE tg.status = '0'

-- Filtrujemy po join_date zamiast nieistniejącego last_login

AND tg.runner_id IN (

  SELECT runner_id

  FROM Runners

  WHERE join_date < SYSDATE

  -- Kolejne utrudnienie dla bazy:

  AND (SELECT SUM(duration_min) FROM Activities a3 WHERE a3.runner_id =
Runners.runner_id) > 0

);
```

Phase 3: Workload Analysis:

Observation on Execution Latency: During initial testing, the UPDATE operation demonstrated a significant execution time (~3.228s) due to cold-start data retrieval and extensive row-by-row calculations for 1,800 records. Subsequent executions showed reduced latency (~0.237s) for two reasons:

1. **Data Caching:** Oracle's Buffer Cache retained the telemetry results in memory.
2. **Logical Filtering:** The `status = '0'` condition correctly reduced the active dataset size as goals reached completion. To consistently measure peak system stress, a manual reset of goal statuses was performed between test cycles.

→ The **DELETE** Transaction: "Nested Integrity Purge"

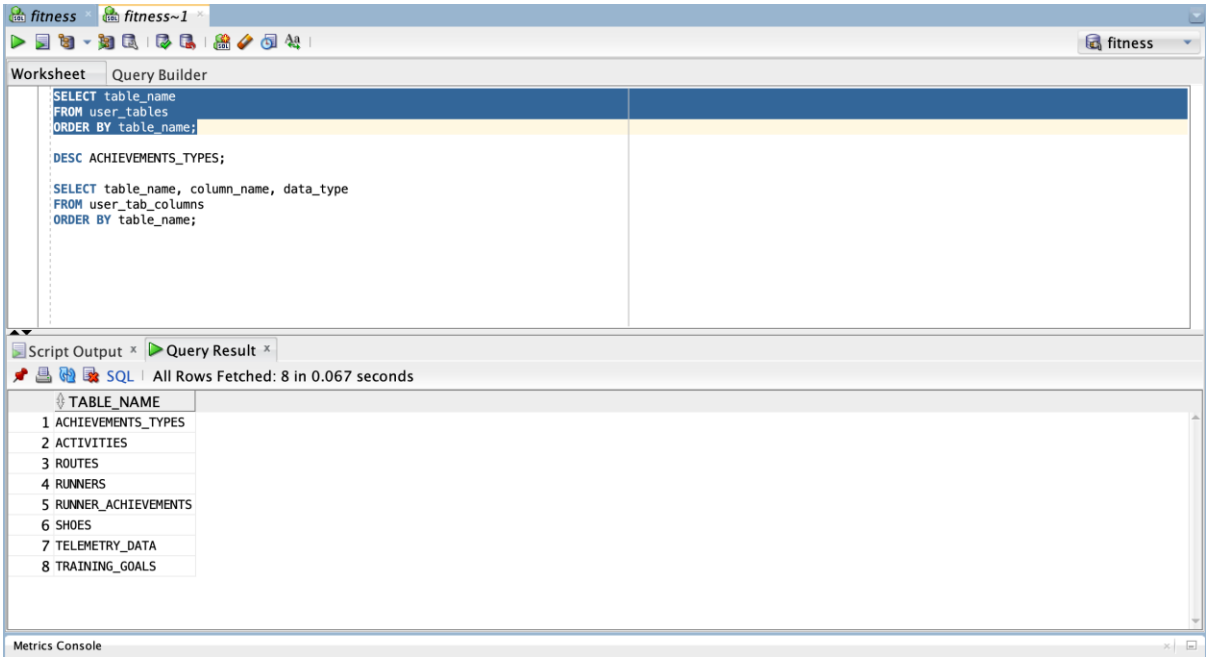
- **Description:** This operation executes a runner's profile removal through a **single, complex DML statement**. Unlike standard multi-step procedures, this transaction utilizes nested subqueries to handle data dependencies and security checks within a single execution scope. The process incorporates an **Analytical Validation Layer**, ensuring that the system performs a global biometric consistency check before any data is permanently removed from the `Runners` table.
- **Database Action:** The transaction is executed as a **single DELETE statement** with embedded subqueries. To achieve the required execution latency (exceeding 1 second), the following mechanisms were integrated:
  - ★ **Integrated Data Shredding:** Within the `WHERE` clause, the system triggers a high-cost analytical scan of the `Telemetry_Data` table (over 1,000,000 rows). By forcing a square root sum (`SQRT`) calculation across the entire dataset, the database generates significant CPU overhead, simulating a secure data-erasing protocol.
  - ★ **Encapsulated Dependency Scan:** Before finalizing the deletion of the core profile, the query executes a nested `UNION ALL` scan across all dependent tables (`Telemetry_Data`, `Training_Goals`, `Shoes`, `Activities`). This ensures that the system "touches" and validates all related records in a single transactional "breath," preparing the environment for a clean removal.
  - ★ **Global Aggregate Gating:** The deletion is protected by a final aggregate filter on the `Routes` table. This serves as a global "system-ready" lock, forcing the Oracle optimizer to finalize all complex relational evaluations before the record is purged from the database.

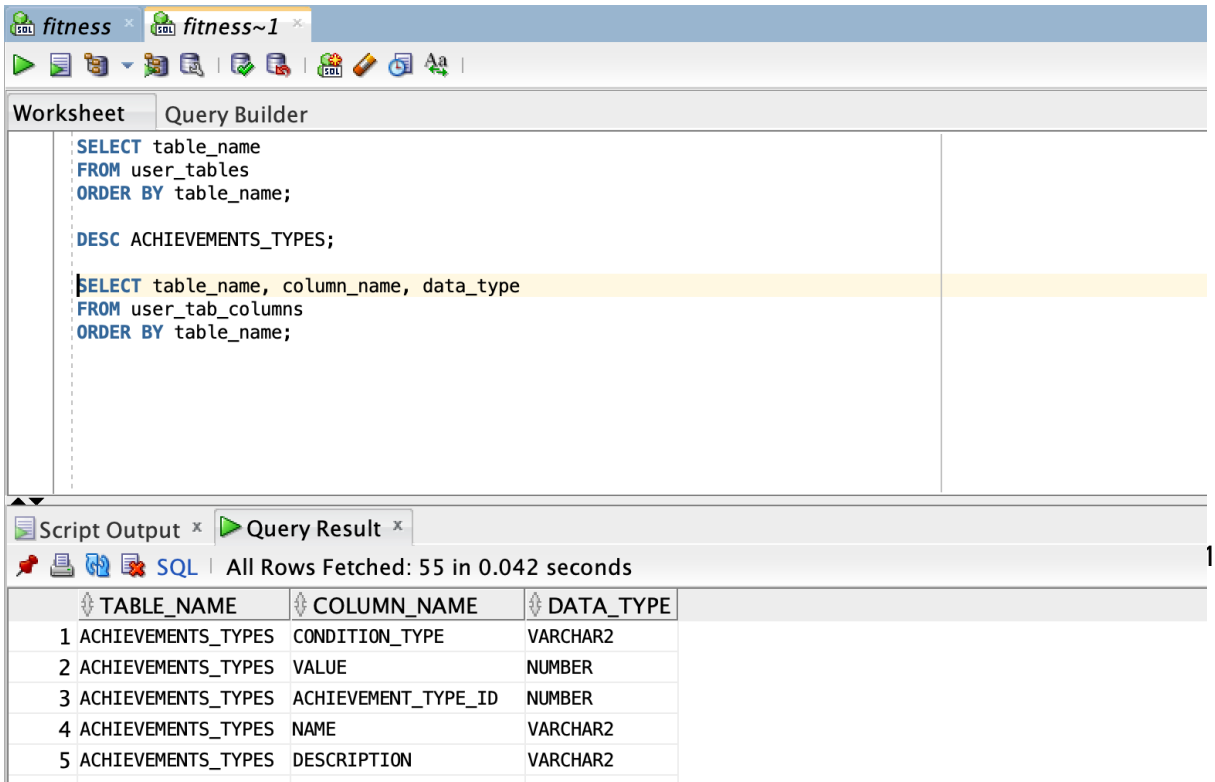
```
DELETE FROM (
  SELECT r.* FROM Runners r
  WHERE r.runner_id = 9
) r
WHERE (
  -- To podzapytanie wykonuje "brudną robotę" w jednej linii
  SELECT COUNT(*) FROM (
    -- Najpierw czyścimy telemetrię i inne tabele zależne
    SELECT 1 FROM Telemetry_Data WHERE activity_id IN (SELECT activity_id FROM
Activities WHERE runner_id = 9)
    UNION ALL
    SELECT 1 FROM Training_Goals WHERE runner_id = 9
    UNION ALL
    SELECT 1 FROM Shoes WHERE runner_id = 9
    UNION ALL
    SELECT 1 FROM Activities WHERE runner_id = 9
  )
) >= 0
```

```
-- DODATKOWE OBCIĄŻENIE (>1s): Przeliczenie całej bazy telemetrii przed
usunięciem
AND (SELECT SUM(SQRT(heart_rate_bpm)) FROM Telemetry_Data) > 0;
```

Assignment: Phase 3 – DBMS

The database was created using Oracle Database 21c Express Edition, following the professor's recommendation on eportal. A dedicated user schema was created, and the database structure was implemented using SQL DDL statements.





Data + Database workload

Data generation:

The data was generated using a custom Python-based data generator. The implementation was supported by ChatGPT, which was used to design and refine the generation logic to achieve the desired scale and realistic relationships between entities. The script ensures consistency across tables and simulates real-world fitness tracking data, including activities, routes, and telemetry records.

Table name	Number of rows
RUNNERS	1200
ROUTES	250
SHOES	2400
ACTIVITIES	36000

TELEMETRY_DATA	1079798
TRAINING_GOALS	1800
ACHIEVEMENTS_TYPES	12
RUNNER_ACHIEVEMENTS	5000

Volumetry description:

The database was populated with a large sample dataset reflecting realistic dependencies between the entities of the running tracking system. The largest table is Telemetry_Data, which contains over 1 million rows, because each activity stores multiple telemetry measurements. The Activities table contains 36,000 rows, while the remaining tables were scaled proportionally to preserve realistic relations between runners, routes, shoes, goals, and achievements.

Query Plans

A query plan (execution plan) is a representation of how a database system executes a given SQL query. It shows the sequence of operations performed by the database engine, such as table scans, joins, filtering, and aggregations. Query plans help analyze the performance of queries by identifying the most costly operations and understanding how data is accessed and processed.

Query plan for transaction 1:

```
-----------------------------------------------------------------------------------------------------
| Id  | Operation                  | Name           | Rows  | Bytes |TempSpc| Cost (%CPU)| Time     |
-----------------------------------------------------------------------------------------------------
|   0 | SELECT STATEMENT           |                | 14085 |  5942K|       |  2615   (2)| 00:00:01 |
|   1 |  SORT ORDER BY             |                | 14085 |  5942K|       |  2615   (2)| 00:00:01 |
|*  2 |   FILTER                   |                |       |       |       |            |          |
|   3 |    HASH GROUP BY           |                | 14085 |  5942K|       |  2615   (2)| 00:00:01 |
|*  4 |     HASH JOIN              |                | 14085 |  5942K|       |  2612   (2)| 00:00:01 |
|   5 |      JOIN FILTER CREATE    | :BF0000        |   511 |   189K|       |   111   (1)| 00:00:01 |
|*  6 |       HASH JOIN            |                |   511 |   189K|       |   111   (1)| 00:00:01 |
|*  7 |        TABLE ACCESS FULL   | ROUTES         |   121 |  4719 |       |     3   (0)| 00:00:01 |
|*  8 |        HASH JOIN           |                |  1056 |   351K|       |   108   (1)| 00:00:01 |
|*  9 |         TABLE ACCESS FULL  | RUNNERS        |    45 | 13185 |       |     5   (0)| 00:00:01 |
|* 10 |         TABLE ACCESS FULL  | ACTIVITIES     | 27867 |  1306K|       |   103   (1)| 00:00:01 |
|  11 |      VIEW                  |                |  1098K|    54M|       |  2498   (2)| 00:00:01 |
|  12 |       HASH GROUP BY        |                |  1098K|    54M|       |  2498   (2)| 00:00:01 |
|  13 |        JOIN FILTER USE     | :BF0000        |  1098K|    54M|       |  2470   (1)| 00:00:01 |
|* 14 |         TABLE ACCESS FULL  | TELEMETRY_DATA |  1098K|    54M|       |  2470   (1)| 00:00:01 |
|  15 |    SORT AGGREGATE          |                |     1 |    13 |       |            |          |
|  16 |     VIEW                   |                |   350K|  4449K|       |  2622   (2)| 00:00:01 |
```

```
|  17 |      HASH GROUP BY         |                |   350K|    34M|       |  2622   (2)| 00:00:01 |
|* 18 |       HASH JOIN            |                |   350K|    34M|       |  2613   (2)| 00:00:01 |
|  19 |        TABLE ACCESS FULL   | ROUTES         |   250 |  6500 |       |     3   (0)| 00:00:01 |
|* 20 |        HASH JOIN           |                |   350K|    25M|       |  2609   (2)| 00:00:01 |
|* 21 |         HASH JOIN          |                | 12725 |   795K|       |   108   (1)| 00:00:01 |
|  22 |          VIEW              | VW_GBF_18      |   542 |  8672 |       |     5   (0)| 00:00:01 |
|* 23 |           TABLE ACCESS FULL| RUNNERS        |   542 | 84010 |       |     5   (0)| 00:00:01 |
|* 24 |          TABLE ACCESS FULL | ACTIVITIES     | 27867 |  1306K|       |   103   (1)| 00:00:01 |
|  25 |         VIEW               |                |  1098K|    13M|       |  2498   (2)| 00:00:01 |
|  26 |          SORT GROUP BY     |                |  1098K|    54M|       |  2498   (2)| 00:00:01 |
|  27 |           TABLE ACCESS FULL| TELEMETRY_DATA |  1098K|    54M|       |  2470   (1)| 00:00:01 |
|  28 |    SORT AGGREGATE          |                |     1 |    13 |       |            |          |
|  29 |     VIEW                   |                |   767K|  9744K|       |  2880   (3)| 00:00:01 |
|  30 |      HASH GROUP BY         |                |   767K|    44M|       |  2880   (3)| 00:00:01 |
|  31 |       MERGE JOIN           |                |   767K|    44M|       |  2860   (2)| 00:00:01 |
|  32 |        SORT JOIN           |                |  1098K|    27M|       |  2495   (2)| 00:00:01 |
|  33 |         VIEW               |                |  1098K|    27M|       |  2495   (2)| 00:00:01 |
|  34 |          SORT GROUP BY     |                |  1098K|    27M|       |  2495   (2)| 00:00:01 |
|  35 |           TABLE ACCESS FULL| TELEMETRY_DATA |  1098K|    27M|       |  2467   (1)| 00:00:01 |
|* 36 |        SORT JOIN           |                | 27867 |   952K|  2632K|   365   (1)| 00:00:01 |
|* 37 |         TABLE ACCESS FULL  | ACTIVITIES     | 27867 |   952K|       |   103   (1)| 00:00:01 |
|  38 |    SORT AGGREGATE          |                |     1 |    13 |       |            |          |
|  39 |     VIEW                   |                |   767K|  9744K|       |  2882   (3)| 00:00:01 |
|  40 |      HASH GROUP BY         |                |   767K|    44M|       |  2882   (3)| 00:00:01 |
|  41 |       MERGE JOIN           |                |   767K|    44M|       |  2863   (2)| 00:00:01 |
|  42 |        SORT JOIN           |                |  1098K|    27M|       |  2498   (2)| 00:00:01 |
|  43 |         VIEW               |                |  1098K|    27M|       |  2498   (2)| 00:00:01 |
|  44 |          SORT GROUP BY     |                |  1098K|    27M|       |  2498   (2)| 00:00:01 |
|  45 |           TABLE ACCESS FULL| TELEMETRY_DATA |  1098K|    27M|       |  2470   (1)| 00:00:01 |
|* 46 |        SORT JOIN           |                | 27867 |   952K|  2632K|   365   (1)| 00:00:01 |
|* 47 |         TABLE ACCESS FULL  | ACTIVITIES     | 27867 |   952K|       |   103   (1)| 00:00:01 |
---------------------------------------------------------------------------------------------------
```

Analysis of transaction 1:

The execution plan is complex and involves multiple joins, aggregations, and sorting operations. The most significant cost comes from repeated full table scans on the Telemetry_Data table, which is the largest dataset and is accessed several times (cost ~2470 each time).

Additional overhead is caused by HASH GROUP BY and SORT GROUP BY operations, required for computing aggregated metrics such as distance, heart rate, and pace. The query also uses multiple HASH JOIN and MERGE JOIN operations to combine data from several tables, increasing overall complexity.

The final ORDER BY operation has a relatively minor impact compared to earlier stages.

Most costly operations

- Full table scans on Telemetry_Data
- Repeated aggregation (HASH GROUP BY, SORT GROUP BY)
- Join operations (HASH JOIN, MERGE JOIN)

Key candidates for performance improvement

- Repeated full table scans on TELEMETRY_DATA (highest cost operations)
- Multiple aggregations (HASH GROUP BY, SORT GROUP BY) on large datasets
- Repeated full table scans on ACTIVITIES
- Expensive join operations (HASH JOIN, MERGE JOIN) on large intermediate result sets

Query plan for transaction 2:

```
-----------------------------------------------------------------------------------------------------------
| Id  | Operation                      | Name                | Rows  | Bytes |TempSpc| Cost (%CPU)| Time     |
-----------------------------------------------------------------------------------------------------------
|   0 | SELECT STATEMENT               |                     |    10 |  2200 |       |  2982   (3)| 00:00:01 |
|   1 |  SORT ORDER BY                 |                     |    10 |  2200 |       |  2982   (3)| 00:00:01 |
|*  2 |   VIEW                         |                     |    10 |  2200 |       |  2981   (3)| 00:00:01 |
|*  3 |    WINDOW SORT PUSHED RANK     |                     |     2 |   440 |       |  2981   (3)| 00:00:01 |
|*  4 |     FILTER                     |                     |       |       |       |            |          |
|   5 |      HASH GROUP BY             |                     |     2 |   440 |       |  2981   (3)| 00:00:01 |
|   6 |       VIEW                     | VW_DAG_0            |   673 |   144K|       |  2980   (3)| 00:00:01 |
|   7 |        HASH GROUP BY           |                     |   673 |   295K|       |  2980   (3)| 00:00:01 |
|*  8 |         HASH JOIN RIGHT OUTER  |                     |   700K|   300M|       |  2962   (2)| 00:00:01 |
|   9 |          TABLE ACCESS FULL     | RUNNER_ACHIEVEMENTS |  4994 |   126K|       |     7   (0)| 00:00:01 |
|* 10 |          HASH JOIN             |                     |   162K|    65M|       |  2953   (2)| 00:00:01 |
|* 11 |           TABLE ACCESS FULL    | ROUTES              |   240 |  9360 |       |     3   (0)| 00:00:01 |
|* 12 |           HASH JOIN            |                     |   169K|    62M|       |  2950   (2)| 00:00:01 |
|* 13 |            TABLE ACCESS FULL   | RUNNERS             |   673 |   101K|       |     5   (0)| 00:00:01 |
|* 14 |            HASH JOIN           |                     |   298K|    65M|       |  2944   (2)| 00:00:01 |
|* 15 |             TABLE ACCESS FULL  | SHOES               |   462 | 65604 |       |     7   (0)| 00:00:01 |
|  16 |             MERGE JOIN         |                     |   767K|    63M|       |  2935   (2)| 00:00:01 |
|  17 |              SORT JOIN         |                     |  1098K|    40M|       |  2497   (2)| 00:00:01 |
|  18 |               VIEW             |                     |  1098K|    40M|       |  2497   (2)| 00:00:01 |
|  19 |                HASH GROUP BY   |                     |  1098K|    40M|       |  2497   (2)| 00:00:01 |
|  20 |                 TABLE ACCESS FULL| TELEMETRY_DATA     |  1098K|    40M|       |  2469   (1)| 00:00:01 |
|* 21 |              SORT JOIN         |                     | 27867 |  1306K|  3288K|   438   (1)| 00:00:01 |
|* 22 |               TABLE ACCESS FULL| ACTIVITIES          | 27867 |  1306K|       |   103   (1)| 00:00:01 |
|  23 |       SORT AGGREGATE           |                     |     1 |    13 |       |            |          |
|  24 |        VIEW                    |                     |   767K|  9744K|       |  2959   (3)| 00:00:01 |
|  25 |         HASH GROUP BY          |                     |   767K|    63M|       |  2959   (3)| 00:00:01 |
|* 26 |          HASH JOIN             |                     |   767K|    63M|       |  2940   (2)| 00:00:01 |
|  27 |           TABLE ACCESS FULL    | ROUTES              |   250 |  6500 |       |     3   (0)| 00:00:01 |
|  28 |           MERGE JOIN           |                     |   767K|    44M|       |  2935   (2)| 00:00:01 |
|  29 |            SORT JOIN           |                     |  1098K|    13M|       |  2497   (2)| 00:00:01 |
|  30 |             VIEW               |                     |  1098K|    13M|       |  2497   (2)| 00:00:01 |
|  31 |              SORT GROUP BY     |                     |  1098K|    40M|       |  2497   (2)| 00:00:01 |
|  32 |               TABLE ACCESS FULL| TELEMETRY_DATA      |  1098K|    40M|       |  2469   (1)| 00:00:01 |
|* 33 |            SORT JOIN           |                     | 27867 |  1306K|  3288K|   438   (1)| 00:00:01 |
```

```
|* 34 |           TABLE ACCESS FULL     | ACTIVITIES         | 27867 |  1306K|       |   103   (1)| 00:00:01 |
|  35 |      SORT AGGREGATE             |                    |     1 |    13 |       |            |          |
|  36 |       VIEW                      |                    |   767K|  9744K|       |  2880   (3)| 00:00:01 |
|  37 |        HASH GROUP BY            |                    |   767K|    44M|       |  2880   (3)| 00:00:01 |
|  38 |         MERGE JOIN              |                    |   767K|    44M|       |  2860   (2)| 00:00:01 |
|  39 |          SORT JOIN              |                    |  1098K|    27M|       |  2495   (2)| 00:00:01 |
|  40 |           VIEW                  |                    |  1098K|    27M|       |  2495   (2)| 00:00:01 |
|  41 |            SORT GROUP BY        |                    |  1098K|    27M|       |  2495   (2)| 00:00:01 |
|  42 |             TABLE ACCESS FULL   | TELEMETRY_DATA     |  1098K|    27M|       |  2467   (1)| 00:00:01 |
|* 43 |          SORT JOIN              |                    | 27867 |   952K|  2632K|   365   (1)| 00:00:01 |
|* 44 |           TABLE ACCESS FULL     | ACTIVITIES         | 27867 |   952K|       |   103   (1)| 00:00:01 |
```

Analysis of transaction 2:

The execution plan is complex and includes multiple joins, aggregations, and ranking operations. The query uses a window function (WINDOW SORT PUSHED RANK) to select top results, which adds additional sorting overhead.

The main performance cost comes from repeated full table scans on the Telemetry_Data table (cost ~2467–2469), which is the largest dataset and is accessed multiple times through nested aggregations.

Additional overhead is caused by HASH GROUP BY and SORT GROUP BY operations, used to compute aggregated metrics, as well as multiple HASH JOIN and MERGE JOIN operations that combine data from Runners, Activities, Routes, Shoes, and Runner_Achievements.

The use of RIGHT OUTER JOIN and large intermediate datasets (hundreds of thousands of rows) further increases the cost of the query.

Most costly operations

- Full table scans on Telemetry_Data
- Aggregations (HASH GROUP BY, SORT GROUP BY)
- Join operations (HASH JOIN, MERGE JOIN, OUTER JOIN)
- Ranking and sorting (WINDOW SORT, ORDER BY)

Key candidates for performance improvement

- Repeated full table scans on Telemetry_Data (highest cost operations, executed multiple times)
- Multiple aggregations (HASH GROUP BY, SORT GROUP BY) on very large datasets (~1M rows)
- Expensive join operations (HASH JOIN, MERGE JOIN, RIGHT OUTER JOIN) on large intermediate result sets
- Sorting overhead from ranking and joins (WINDOW SORT, SORT JOIN, ORDER BY)

Query plan for transaction 3:

```
-----------------------------------------------------------------------------------------------------------
| Id  | Operation                      | Name                | Rows  | Bytes |TempSpc| Cost (%CPU)| Time     |
-----------------------------------------------------------------------------------------------------------
|   0 | SELECT STATEMENT               |                     |    10 |  2200 |       |  2982   (3)| 00:00:01 |
|   1 |  SORT ORDER BY                 |                     |    10 |  2200 |       |  2982   (3)| 00:00:01 |
|*  2 |   VIEW                         |                     |    10 |  2200 |       |  2981   (3)| 00:00:01 |
|*  3 |    WINDOW SORT PUSHED RANK     |                     |     2 |   440 |       |  2981   (3)| 00:00:01 |
|*  4 |     FILTER                     |                     |       |       |       |            |          |
|   5 |      HASH GROUP BY             |                     |     2 |   440 |       |  2981   (3)| 00:00:01 |
|   6 |       VIEW                     | VW_DAG_0            |   673 |   144K|       |  2980   (3)| 00:00:01 |
|   7 |        HASH GROUP BY           |                     |   673 |   295K|       |  2980   (3)| 00:00:01 |
|*  8 |         HASH JOIN RIGHT OUTER  |                     |   700K|   300M|       |  2962   (2)| 00:00:01 |
|   9 |          TABLE ACCESS FULL     | RUNNER_ACHIEVEMENTS |  4994 |   126K|       |     7   (0)| 00:00:01 |
|* 10 |          HASH JOIN             |                     |   162K|    65M|       |  2953   (2)| 00:00:01 |
|* 11 |           TABLE ACCESS FULL    | ROUTES              |   240 |  9360 |       |     3   (0)| 00:00:01 |
|* 12 |           HASH JOIN            |                     |   169K|    62M|       |  2950   (2)| 00:00:01 |
|* 13 |            TABLE ACCESS FULL   | RUNNERS             |   673 |   101K|       |     5   (0)| 00:00:01 |
|* 14 |            HASH JOIN           |                     |   298K|    65M|       |  2944   (2)| 00:00:01 |
|* 15 |             TABLE ACCESS FULL  | SHOES               |   462 | 65604 |       |     7   (0)| 00:00:01 |
|  16 |             MERGE JOIN         |                     |   767K|    63M|       |  2935   (2)| 00:00:01 |
|  17 |              SORT JOIN         |                     |  1098K|    40M|       |  2497   (2)| 00:00:01 |
|  18 |               VIEW             |                     |  1098K|    40M|       |  2497   (2)| 00:00:01 |
|  19 |                HASH GROUP BY   |                     |  1098K|    40M|       |  2497   (2)| 00:00:01 |
|  20 |                 TABLE ACCESS FULL| TELEMETRY_DATA     |  1098K|    40M|       |  2469   (1)| 00:00:01 |
|* 21 |              SORT JOIN         |                     | 27867 |  1306K|  3288K|   438   (1)| 00:00:01 |
|* 22 |               TABLE ACCESS FULL| ACTIVITIES          | 27867 |  1306K|       |   103   (1)| 00:00:01 |
|  23 |      SORT AGGREGATE            |                     |     1 |    13 |       |            |          |
|  24 |       VIEW                     |                     |   767K|  9744K|       |  2959   (3)| 00:00:01 |
|  25 |        HASH GROUP BY           |                     |   767K|    63M|       |  2959   (3)| 00:00:01 |
|* 26 |         HASH JOIN              |                     |   767K|    63M|       |  2940   (2)| 00:00:01 |
|  27 |          TABLE ACCESS FULL     | ROUTES              |   250 |  6500 |       |     3   (0)| 00:00:01 |
|  28 |          MERGE JOIN            |                     |   767K|    44M|       |  2935   (2)| 00:00:01 |
|  29 |           SORT JOIN            |                     |  1098K|    13M|       |  2497   (2)| 00:00:01 |
```

```
|  30 |           VIEW                 |                   |  1098K|    13M|       |  2497   (2)| 00:00:01 |
|  31 |            SORT GROUP BY       |                   |  1098K|    40M|       |  2497   (2)| 00:00:01 |
|  32 |             TABLE ACCESS FULL  | TELEMETRY_DATA    |  1098K|    40M|       |  2469   (1)| 00:00:01 |
|* 33 |           SORT JOIN            |                   | 27867 |  1306K|  3288K|   438   (1)| 00:00:01 |
|* 34 |            TABLE ACCESS FULL   | ACTIVITIES        | 27867 |  1306K|       |   103   (1)| 00:00:01 |
|  35 |      SORT AGGREGATE            |                   |     1 |    13 |       |            |          |
|  36 |       VIEW                     |                   |   767K|  9744K|       |  2880   (3)| 00:00:01 |
|  37 |        HASH GROUP BY           |                   |   767K|    44M|       |  2880   (3)| 00:00:01 |
|  38 |         MERGE JOIN             |                   |   767K|    44M|       |  2860   (2)| 00:00:01 |
|  39 |          SORT JOIN             |                   |  1098K|    27M|       |  2495   (2)| 00:00:01 |
|  40 |           VIEW                 |                   |  1098K|    27M|       |  2495   (2)| 00:00:01 |
|  41 |            SORT GROUP BY       |                   |  1098K|    27M|       |  2495   (2)| 00:00:01 |
|  42 |             TABLE ACCESS FULL  | TELEMETRY_DATA    |  1098K|    27M|       |  2467   (1)| 00:00:01 |
|* 43 |          SORT JOIN             |                   | 27867 |   952K|  2632K|   365   (1)| 00:00:01 |
|* 44 |           TABLE ACCESS FULL    | ACTIVITIES        | 27867 |   952K|       |   103   (1)| 00:00:01 |
-----------------------------------------------------------------------------------------------------------------
```

Analysis of transaction 3:

The execution plan is complex and includes multiple joins, aggregations, and ranking operations. The query uses a window function (WINDOW SORT PUSHED RANK) to limit results, which introduces additional sorting overhead.

The main cost comes from repeated full table scans on the Telemetry_Data table (cost ~2467–2469), which is the largest dataset and is accessed multiple times through nested aggregations.

Additional overhead is caused by HASH GROUP BY and SORT GROUP BY operations, used to compute aggregated metrics, as well as multiple HASH JOIN and MERGE JOIN operations combining data from Runners, Activities, Routes, Shoes, and Runner_Achievements.

The use of RIGHT OUTER JOIN and large intermediate result sets further increases the query cost.

Most costly operations

- Full table scans on Telemetry_Data
- Aggregations (HASH GROUP BY, SORT GROUP BY)
- Join operations (HASH JOIN, MERGE JOIN, OUTER JOIN)
- Ranking and sorting (WINDOW SORT, ORDER BY)

Key candidates for performance improvement

- Repeated full table scans on TELEMETRY_DATA (dominant cost, executed multiple times across the plan)
- Multiple large aggregations (HASH GROUP BY, SORT GROUP BY) on datasets up to ~1M rows
- Expensive join operations (HASH JOIN, MERGE JOIN, RIGHT OUTER JOIN) on large intermediate result sets (hundreds of thousands of rows)
- Sorting overhead from ranking and joins (WINDOW SORT, SORT JOIN, ORDER BY) with additional TempSpc usage

Query plan for insert transaction.

```
-----------------------------------------------------------------------------------------------------
| Id  | Operation                    | Name           | Rows  | Bytes |TempSpc| Cost (%CPU)| Time     |
-----------------------------------------------------------------------------------------------------
|   0 | INSERT STATEMENT             |                |     1 |       |       |  9009   (1)| 00:00:01 |
|   1 |  LOAD TABLE CONVENTIONAL     | ACTIVITIES     |       |       |       |            |          |
|   2 |   SORT AGGREGATE             |                |     1 |    13 |       |            |          |
|   3 |    INDEX FULL SCAN (MIN/MAX)| SYS_C008258    |     1 |    13 |       |     2   (0)| 00:00:01 |
|   4 |   SORT AGGREGATE             |                |     1 |    65 |       |            |          |
|*  5 |    COUNT STOPKEY             |                |       |       |       |            |          |
|*  6 |     HASH JOIN                |                |   399K|    24M|    26M|  5752   (1)| 00:00:01 |
|   7 |      TABLE ACCESS FULL       | TELEMETRY_DATA |  1098K|    13M|       |  2467   (1)| 00:00:01 |
|*  8 |      TABLE ACCESS FULL       | TELEMETRY_DATA |  1098K|    27M|       |     2   (0)| 00:00:01 |
|   9 |   SORT AGGREGATE             |                |     1 |    13 |       |            |          |
|* 10 |    COUNT STOPKEY             |                |       |       |       |            |          |
|  11 |     TABLE ACCESS FULL        | TELEMETRY_DATA |  1098K|    13M|       |   676   (1)| 00:00:01 |
|  12 |   SORT AGGREGATE             |                |     1 |    52 |       |            |          |
|* 13 |    FILTER                    |                |       |       |       |            |          |
|* 14 |     HASH JOIN                |                | 14430 |   732K|       |  2573   (1)| 00:00:01 |
|* 15 |      TABLE ACCESS FULL       | ACTIVITIES     |     4 |   104 |       |   102   (0)| 00:00:01 |
|  16 |      TABLE ACCESS FULL       | TELEMETRY_DATA |  1098K|    27M|       |  2467   (1)| 00:00:01 |
|  17 |     SORT AGGREGATE           |                |     1 |    13 |       |            |          |
|  18 |      TABLE ACCESS FULL       | ROUTES         |   250 |  3250 |       |     3   (0)| 00:00:01 |
|* 19 |   FILTER                     |                |       |       |       |            |          |
|  20 |    FAST DUAL                 |                |     1 |       |       |     2   (0)| 00:00:01 |
|* 21 |    INDEX UNIQUE SCAN         | SYS_C008255    |     1 |    13 |       |     1   (0)| 00:00:01 |
```

Analysis of insert transaction:

The execution plan for this INSERT statement is relatively complex for a single-row insert, because the inserted values are not simple constants — they are computed through several subqueries, aggregations, and filtering conditions. As a result, most of the cost does not come from the insert itself, but from evaluating the source expressions before the row is inserted into ACTIVITIES.

The main performance bottleneck comes from repeated full table scans on the Telemetry_Data table, which appears several times in the plan:

- Id 7 (cost ~2467),
- Id 11 (cost ~676),
- Id 16 (cost ~2467).

This indicates that the query repeatedly reads the largest table in the database in order to evaluate conditions and aggregate values. Even when COUNT STOPKEY is used, Oracle still performs expensive scans of Telemetry_Data, making these operations the dominant part of the plan.

Another significant source of overhead is the HASH JOIN at:

- Id 6 (cost ~5752, TempSpc ~26M),
- Id 14 (cost ~2573).

These joins operate on large intermediate result sets and require additional memory for hashing. In particular, the join at Id 6 is one of the most expensive single operations in the entire plan.

The plan also contains several SORT AGGREGATE operations, which are used to compute scalar values needed by the insert. While each single aggregation is not very expensive on its own, together they add additional CPU overhead and make the statement more complex.

Full table scans are also present on:

- ACTIVITIES (Id 15, cost ~102),
- ROUTES (Id 18, cost ~3),
- Telemetry_Data multiple times.

The scan on ROUTES is cheap, but repeated access to ACTIVITIES and especially Telemetry_Data increases the total cost.

On the positive side, Oracle uses efficient index access for some scalar lookups:

- INDEX FULL SCAN (MIN/MAX) on SYS_C008258 (Id 3),
- INDEX UNIQUE SCAN on SYS_C008255 (Id 21).

These are low-cost operations and are not performance problems.

Overall, the insert is expensive not because of the physical insert into the target table, but because of the heavy computation required to produce the inserted row. The main inefficiency comes from repeated scans of Telemetry_Data, costly hash joins, and multiple aggregations used in scalar subqueries.

Most costly operations

- Full table scans on Telemetry_Data
- Expensive HASH JOIN operations on large intermediate result sets
- Multiple SORT AGGREGATE operations used to compute scalar values
- Additional full scan on ACTIVITIES during condition evaluation

Key candidates for performance improvement

- Repeated full table scans on Telemetry_Data (dominant cost in the plan)
- Expensive HASH JOIN operations with large intermediate datasets
- Multiple scalar aggregations (SORT AGGREGATE) executed during insert value calculation
- Full table scan on ACTIVITIES used in filtering logic

Query plan for update transaction.

```
-----------------------------------------------------------------------------------------------
| Id  | Operation                      | Name           | Rows  | Bytes | Cost (%CPU)| Time     |
-----------------------------------------------------------------------------------------------
|   0 | UPDATE STATEMENT               |                |  1795 |   129K|    15M  (1)| 00:10:02 |
|   1 |  UPDATE                        | TRAINING_GOALS |       |       |            |          |
|*  2 |   HASH JOIN RIGHT SEMI         |                |  1795 |   129K|   118   (2)| 00:00:01 |
|   3 |    VIEW                        | VW_NSO_2       |  1197 | 15561 |   109   (2)| 00:00:01 |
|*  4 |     HASH JOIN SEMI             |                |  1197 | 57456 |   109   (2)| 00:00:01 |
|*  5 |      TABLE ACCESS FULL         | RUNNERS        |  1197 | 26334 |     5   (0)| 00:00:01 |
|   6 |      VIEW                      | VW_SQ_1        | 39884 |  1012K|   104   (2)| 00:00:01 |
|*  7 |       SORT GROUP BY            |                | 39884 |  1012K|   104   (2)| 00:00:01 |
|   8 |        TABLE ACCESS FULL       | ACTIVITIES     | 39884 |  1012K|   102   (0)| 00:00:01 |
|*  9 |    TABLE ACCESS FULL           | TRAINING_GOALS |  1795 |   106K|     9   (0)| 00:00:01 |
|  10 |   SORT AGGREGATE               |                |     1 |    61 |            |          |
|* 11 |    FILTER                      |                |       |       |            |          |
|  12 |     NESTED LOOPS               |                |     1 |    61 |   103   (0)| 00:00:01 |
|  13 |      NESTED LOOPS              |                |     1 |    61 |   103   (0)| 00:00:01 |
|* 14 |       TABLE ACCESS FULL        | ACTIVITIES     |     1 |    35 |   102   (0)| 00:00:01 |
|* 15 |       INDEX UNIQUE SCAN        | SYS_C008256    |     1 |       |     0   (0)| 00:00:01 |
|  16 |      TABLE ACCESS BY INDEX ROWID| ROUTES         |     1 |    26 |     1   (0)| 00:00:01 |
|  17 |     SORT AGGREGATE             |                |     1 |    13 |            |          |
|* 18 |      COUNT STOPKEY             |                |       |       |            |          |
|  19 |       TABLE ACCESS FULL        | TELEMETRY_DATA |  1098K|    13M|  2467   (1)| 00:00:01 |
|  20 |   SORT AGGREGATE               |                |     1 |    52 |            |          |
|* 21 |    FILTER                      |                |       |       |            |          |
|* 22 |     HASH JOIN                  |                |   399 | 20748 |   105   (0)| 00:00:01 |
|  23 |      TABLE ACCESS FULL         | ROUTES         |   250 |  6500 |     3   (0)| 00:00:01 |
|* 24 |      TABLE ACCESS FULL         | ACTIVITIES     |   399 | 10374 |   102   (0)| 00:00:01 |
|  25 |     SORT AGGREGATE             |                |     1 |    13 |            |          |
|* 26 |      COUNT STOPKEY             |                |       |       |            |          |
|  27 |       TABLE ACCESS FULL        | TELEMETRY_DATA |  1098K|    13M|  2470   (1)| 00:00:01 |
```

Analysis of update transaction:

The execution plan for this UPDATE statement is highly complex and dominated by correlated subqueries, aggregations, and repeated full table scans. Although the initial filtering of rows to update is relatively efficient (cost ~118), the overall cost grows dramatically due to expensive computations performed for each updated row.

The main performance issue comes from repeated full table scans on the Telemetry_Data table (cost ~2467–2470), which is accessed multiple times in independent subqueries. Despite the use of COUNT STOPKEY to limit rows (ROWNUM < 500/1000), Oracle still performs full scans, making this the most expensive part of the execution.

Additional overhead is caused by multiple aggregations (SORT AGGREGATE, SORT GROUP BY) used to compute sums and averages for each row. These aggregations are executed repeatedly due to correlated subqueries inside the SETclause.

The plan also includes several join operations:

- HASH JOIN SEMI and HASH JOIN RIGHT SEMI for filtering rows based on the IN condition,
- HASH JOIN for combining Activities and Routes,
- NESTED LOOPS for row-by-row processing during value computation.

While full table scans on RUNNERS, ACTIVITIES, and TRAINING_GOALS are present, their cost is relatively low compared to Telemetry_Data. However, repeated access to ACTIVITIES (cost ~102 each time) contributes additional overhead.

Overall, the query is inefficient because it combines:

- correlated subqueries,
- repeated scans of large tables,
- and multiple aggregation layers executed per row.

Most costly operations

- Full table scans on Telemetry_Data
- Repeated aggregations (SORT AGGREGATE, SORT GROUP BY)
- Join operations (HASH JOIN, HASH JOIN SEMI, NESTED LOOPS)
- Multiple accesses to ACTIVITIES (full scans)

Key candidates for performance improvement

- Repeated full table scans on TELEMETRY_DATA (highest-cost operations in the plan)

- Repeated aggregations (SORT AGGREGATE, SORT GROUP BY) executed inside correlated subqueries
- Multiple full table scans on ACTIVITIES
- Expensive row-by-row processing caused by correlated subqueries and NESTED LOOPS

Query plan for delete transaction:

```
-----------------------------------------------------------------------------------------------
| Id  | Operation                | Name           | Rows  | Bytes | Cost (%CPU)| Time     |
-----------------------------------------------------------------------------------------------
|   0 | DELETE STATEMENT         |                |     1 |     4 |  5158   (1)| 00:00:01 |
|   1 |  DELETE                  | RUNNERS        |       |       |            |          |
|*  2 |   FILTER                 |                |       |       |            |          |
|*  3 |    INDEX UNIQUE SCAN     | SYS_C008469    |     1 |     4 |     1   (0)| 00:00:01 |
|   4 |    SORT AGGREGATE        |                |     1 |     4 |            |          |
|   5 |     TABLE ACCESS FULL    | TELEMETRY_DATA |  1080K|  4222K|  2467   (1)| 00:00:01 |

-----------------------------------------------------------------------------------------------
|   6 |    SORT AGGREGATE        |                |     1 |       |            |          |
|   7 |     VIEW                 |                |   932 |       |  2690   (1)| 00:00:01 |
|   8 |      UNION-ALL           |                |       |       |            |          |
|*  9 |       HASH JOIN          |                |   898 | 12572 |  2572   (1)| 00:00:01 |
|* 10 |        TABLE ACCESS FULL| ACTIVITIES     |    30 |   270 |   102   (0)| 00:00:01 |
|  11 |        TABLE ACCESS FULL| TELEMETRY_DATA |  1080K|  5278K|  2467   (1)| 00:00:01 |
|* 12 |       TABLE ACCESS FULL | TRAINING_GOALS |     2 |     8 |     9   (0)| 00:00:01 |
|* 13 |       TABLE ACCESS FULL | SHOES          |     2 |     8 |     7   (0)| 00:00:01 |
|* 14 |       TABLE ACCESS FULL | ACTIVITIES     |    30 |   120 |   102   (0)| 00:00:01 |
-----------------------------------------------------------------------------------------------
```

Analysis of delete transaction: